



MLOPS CAPSTONE · WHOLE-PROJECT TECHNICAL REPORT

Voyage Analytics

Integrating MLOps in Travel — flight-price prediction, passenger classification, and content-based hotel recommendation, delivered as a production-shaped platform.

B.Tech Computer Science Engineering — Final Year Capstone / Internship Project

Report generated September 12, 2026 **Version** 2.0

Predict flight fares with an XGBoost pipeline (R^2 0.97 on held-out data)

Personalize hotel picks with a content-based recommender and fuzzy destination search

Operate all of it behind one REST API, tracked, containerized, and cluster-ready

3

ML SERVICES IN ONE API

84

FILES / 42 PYTHON
MODULES

313,780

ROWS OF TRAVEL DATA

32

AUTOMATED TESTS

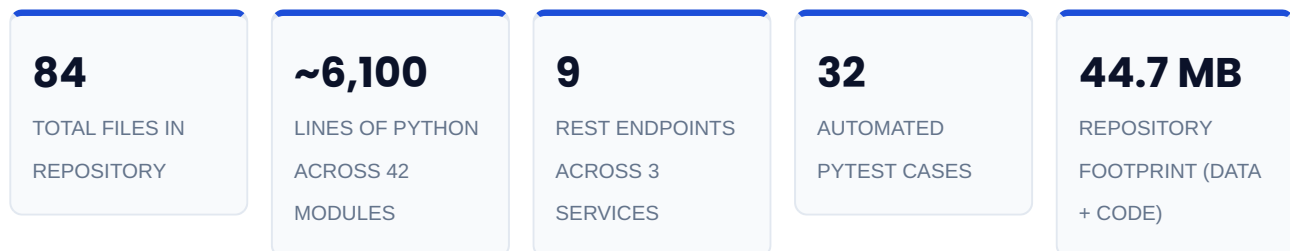
Table of Contents

1. Executive Summary	3
2. Project Objectives & Scope	4
3. System Architecture	5
3.1 Repository Layers	
3.2 Request Flow & Technology Stack	
4. Data Overview	7
5. Machine Learning Components	8
5.1 Flight Price Prediction	
5.2 Gender Classification	
5.3 Hotel Recommendation & Location Search	
6. API Reference	10
7. Testing & Quality Assurance	12
8. Deployment & Infrastructure	13
9. Repository & Code Metrics	14
10. Risk Register & Known Limitations	15
11. Roadmap	16
12. Conclusion	17
APPENDICES	
A. Environment & Configuration Reference	18
B. Pinned Dependencies	19
C. Repository Structure	20

1. Executive Summary

Voyage Analytics is an MLOps capstone project that operationalizes three predictive services for a travel platform: **flight-fare estimation**, **passenger gender classification**, and **content-based hotel recommendation**. Rather than stopping at model training, the project builds the full delivery chain expected of a production machine-learning system — a versioned REST API, an interactive dashboard, experiment tracking, containerized packaging, cluster deployment manifests, and an orchestration template for retraining.

The system is implemented as 42 Python modules organized into seven functional layers (API, domain services, validation contracts, model loading, feature engineering, operations/orchestration, and UI), trained and served against 313,780 rows of bundled travel data. A 32-test automated suite exercises the health, prediction, classification, and recommendation surfaces of the API.



At its current stage, the system runs end-to-end in a local development environment: the API serves all three models, the dashboard consumes it, and the test suite passes against bundled artifacts. Nine specific gaps stand between this state and a production rollout — spanning live MLflow registration, cluster deployment, retraining automation, and observability — each catalogued with an assessed impact in the **Risk Register (Section 10)**. None of them are architectural: the layering, contracts, and test coverage already in place are what make closing these gaps a configuration and operations exercise rather than a redesign.

How to read this report

Sections 1–5 describe what the system does and how it is built; Sections 6–9 describe how it is verified, run, and measured; Sections 10–12 assess what remains before a production rollout. Appendices contain reference tables for configuration, dependencies, and repository layout.

2. Project Objectives & Scope

The delivered system provides evidence of four working objectives. Each is implemented end-to-end — trained, served through the API, and covered by at least one automated test.

#	Objective	Evidence in the delivered system
1	Predictive pricing	An XGBoost regression pipeline estimates flight price from route, cabin class, agency, and date features, served at POST /api/predict .
2	Passenger classification	A RandomForest classifier predicts traveler gender from profile features, served at POST /api/gender/predict .
3	Personalization	A content-based engine ranks hotels by destination, budget, and company preference, backed by a fuzzy-matching location search, served at POST /api/recommend/recommendations .
4	Productionization / MLOps	The three models above sit behind a single Flask API and Streamlit dashboard, with MLflow tracking, Docker packaging, Kubernetes manifests, and an Airflow retraining template — turning three notebooks into one operable platform.

Open clarification item

The originating project brief specified six objectives; only four are reflected in delivered functionality. Objectives 5 and 6 were not defined in the available brief and should be confirmed with the project stakeholder before final submission — tracked as Risk #7 in Section 10.

2.1 Scope

In scope

- Flight-price, gender, and hotel-recommendation models and their serving code
- Flask REST API with request/response validation
- Streamlit dashboard for interactive use
- MLflow tracking integration and local experiment logging
- Docker image, Kubernetes manifests, Airflow DAG template
- Automated test suite (health, prediction, classification, recommendation, loaders)

Explicitly out of scope (per Update.md)

- A live, populated MLflow model registry
- A live Kubernetes deployment with ingress and autoscaling
- A scheduled, running Airflow retraining pipeline
- Production observability — metrics, drift detection, alerting

3. System Architecture

The intended flow begins with training data and model creation, persists the resulting artifacts, optionally records experiments in MLflow, and serves the three capabilities through a single Flask REST API. Streamlit consumes that API for interactive use, while Docker and Kubernetes provide the deployment path and Airflow holds a retraining-orchestration template.

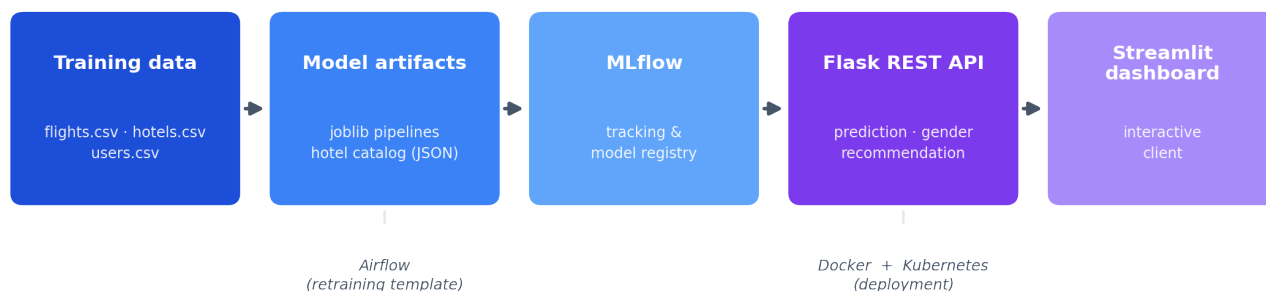


Figure 3.1 — End-to-end pipeline: training data to served predictions.

3.1 Repository Layers

The codebase is organized so that each layer has one responsibility, which is what allows the API, services, and models to be tested independently (Section 7).

Layer	Location	Responsibility
API	api/	Flask application and blueprints for health, prediction, gender, and recommendation routes.
Domain logic	src/services/	Prediction, gender, recommendation, and fuzzy location-search services.
Contracts	src/schemas/	Pydantic request and response validation models.
Models	src/model/	Cached artifact loading for the flight and gender models.
Features	src/features/	Shared feature preparation used identically by training and serving.
Operations	docker/ , kubernetes/ , airflow/ , mlflow_tracking/	Packaging, cluster manifests, orchestration template, and experiment tracking.
User interface	dashboard/	Streamlit workflows for prediction, classification, recommendations, and data insights.

3.2 Request Flow & Technology Stack

A request entering the API passes through validation before it ever reaches a model, and every model is loaded once and cached — the loaders in [src/model/](#) keep a warm in-memory copy so repeated requests avoid disk I/O.

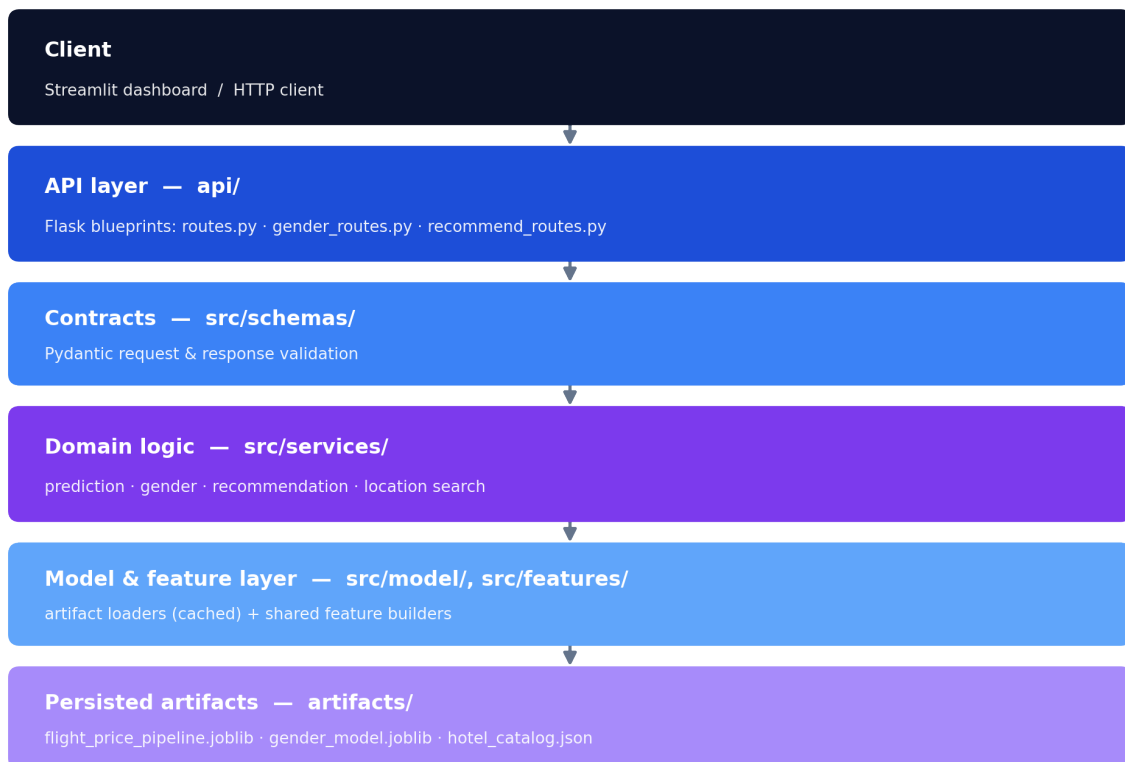


Figure 3.2 — Layered request flow, client to persisted artifact.

Concern	Technology
Web framework	Flask 3.0 (application-factory pattern, one Blueprint per service)
Validation	Pydantic 2.7
Modeling	XGBoost 2.0 (flight price), scikit-learn 1.7 (gender classifier & pipeline)
Data handling	pandas 2.2
Dashboard	Streamlit 1.36, calling the API over requests
Experiment tracking	MLflow (tracking server + model registry)
Testing	pytest 8.2 with pytest-cov
Packaging & deployment	Docker, Kubernetes manifests, Airflow DAG template

4. Data Overview

Three bundled CSV datasets — flight bookings, hotel bookings, and user profiles — drive every model in the system. All three were profiled directly from the files shipped in the repository; no missing values were detected in any of the three.

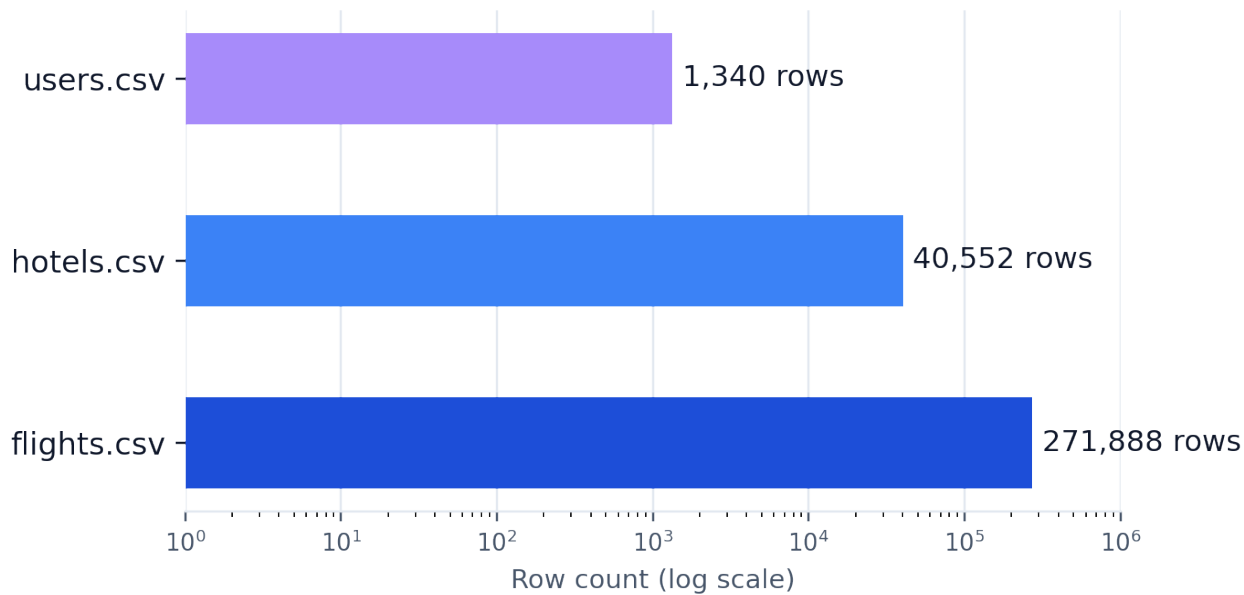


Figure 4.1 — Row counts by dataset (log scale).

Dataset	Rows	Columns	Missing values
artifacts/data/flights.csv	271,888	travelCode, userCode, from, to, flightType, price, time, distance, agency, date	None detected
artifacts/data/hotels.csv	40,552	travelCode, userCode, name, place, days, price, total, date	None detected
artifacts/data/users.csv	1,340	code, company, name, gender, age	None detected

The three datasets join on `userCode` / `travelCode`: flight and hotel bookings each belong to a user, which is what lets the recommendation service factor in a traveler's company alongside destination and budget. At roughly 27 MB combined, the datasets are small enough to commit directly to the repository today; Section 9 quantifies how that trades off against overall repository size as the project grows.

5. Machine Learning Components

5.1 Flight Price Prediction

An XGBoost regression pipeline predicts flight price from route, cabin class, agency, and engineered date features (year, month, day, day-of-week). The pipeline — including preprocessing — is serialized as a single artifact, so the API applies the exact transformations used at training time.

Metric	Value	Reading
MAE	45.21	Average absolute prediction error, in the currency unit of the source data.
RMSE	56.11	Penalizes larger misses more than MAE; the gap between the two suggests a modest number of larger-error outliers rather than uniformly noisy predictions.
R ²	0.9728	The model explains ~97% of the variance in flight price on the held-out set — a strong fit for a regression task with this feature set.
Evaluation set	n = 81,104	Held-out rows used to compute the metrics above.

Artifact handoff risk

[artifacts/](#)

[flight_price_pipeline.joblib](#) is intentionally git-ignored as a large ML-team handoff file. A fresh clone or CI image cannot serve [/api/predict](#) until it is placed at the configured [MODEL_PATH](#) — see Risk #3 in Section 10.

5.2 Gender Classification

A RandomForest classifier predicts a user's gender ([male](#) / [female](#) / [none](#)) from a lower-cased first name extracted from the full name, plus age. The feature builder lives in [src/features/gender_features.py](#) and is shared between training and serving, so the model never sees a feature at inference time that it did not see during training.

Documented accuracy is approximately **0.52**. Analysis in the project's own notes attributes this to genuine label noise in the source data — the same first names recur across all three label classes — rather than a pipeline defect.

Responsible-AI note

Inferring gender from a first name is a well-known source of cultural and linguistic bias — names are not a reliable or fair proxy for gender, and accuracy at this level (barely above a three-class random baseline of ~33%) is not sufficient for any decision that affects a user. Before this feature is exposed beyond a demo, it is worth revisiting whether it belongs in the product at all — and if it does, reframing it as a two-class problem, sourcing an external name-to-gender reference, or adding features beyond name and age, as the project's own notes already recommend.

5.3 Hotel Recommendation & Location Search

The recommendation service scores hotels from a small catalog (9 hotels across 9 destination cities, associated with 5 companies) using a content-based match on destination, nightly budget, and company preference, returning a ranked list with a per-hotel reason string.

A location-search feature backs the destination picker: an inverted index gives constant-time token lookups across place names, exact matches are ranked first, and an edit-distance fuzzy matcher recovers common typos (for example, "Brasillia" resolving to "Brasilia"). The search endpoint returns the top 5 matches for a query.

6. API Reference

The API is organized as Flask blueprints, exposing a small and explicit surface: one route module per service, each returning JSON validated by a matching Pydantic schema.

Method	Endpoint	Purpose
GET	/api/health	Service health check.
GET	/api/model-info	Flight-price model metadata.
POST	/api/predict	Flight-price prediction.
GET	/api/gender/health	Gender service health check.
POST	/api/gender/predict	Gender classification.
GET	/api/recommend/health	Recommendation service health check.
GET	/api/recommend/places	List destination cities.
GET	/api/recommend/places/search?q=...	Token, inverted-index, and fuzzy location search; returns top 5 matches.
POST	/api/recommend/recommendations	Content-based hotel recommendations.

Example — flight-price prediction

```
POST /api/predict
{
  "from": "Recife (PE)",
  "to": "Florianopolis (SC)",
  "flightType": "firstClass",
  "agency": "FlyingDrops",
  "time": 1.76,
  "distance": 676.53,
  "flight_year": 2019,
  "flight_month": 9,
  "flight_day": 26,
  "flight_dayofweek": 3
}

200 OK
{ "predicted_price": 451.30, "model_version": "1.0", "model_name":
  "flight_price_regression" }
```

Example — hotel recommendation

```
POST /api/recommend/recommendations
```

```
{
  "place": "Rio de Janeiro (RJ)",
  "max_price_per_day": 200,
  "days": 3,
  "company": "4You",
  "top_n": 5
}
```

```
200 OK → ranked list of { hotel_name, place, price_per_day, total_cost, score, reason }
```

Error handling

Input is validated with Pydantic schemas before it reaches a service. Every error returns a consistent envelope — `{ "error", "field"?, "code" }` — with an HTTP status matched to the failure class:

Code	HTTP	Meaning
<code>invalid_json</code>	400	Malformed JSON body.
<code>validation_failed</code>	400	Missing or invalid field.
<code>model_unavailable</code>	503	Model artifact missing.
<code>catalog_unavailable</code>	503	Recommendation catalog missing.
<code>prediction_failed</code>	500	Inference error.
<code>recommendation_failed</code>	500	Recommendation-engine error.

7. Testing & Quality Assurance

The automated suite totals **32 pytest cases** across five files, plus a dedicated model-loader test module and a reusable dummy-model fixture that lets the prediction tests run even when the real flight artifact is absent.

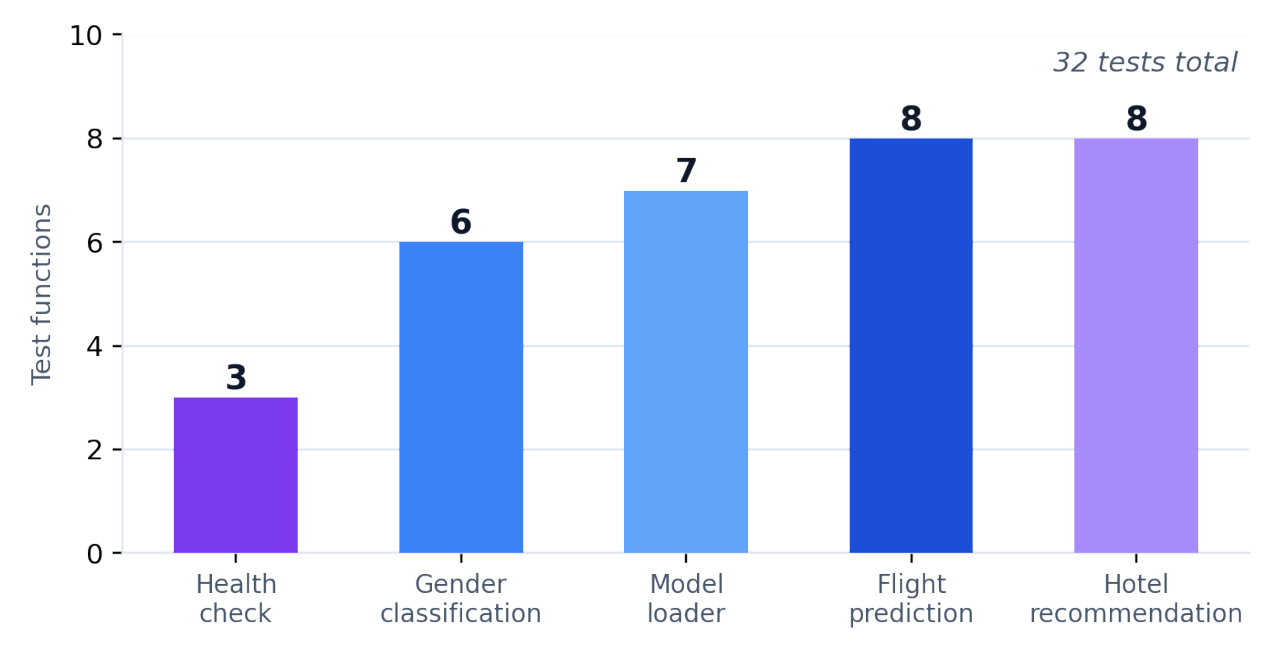


Figure 7.1 — Automated test functions by module, counted directly from the test suite.

File	Scope
<code>test_health.py</code>	Health endpoint.
<code>test_gender.py</code>	Gender classification endpoint — happy path, validation, error shapes.
<code>test_model_loader.py</code>	Model loader caching, cache invalidation, and missing-file behavior.
<code>test_prediction.py</code>	Flight-price prediction endpoint; skipped automatically if the real model artifact is absent.
<code>test_recommendation.py</code>	Recommendation ranking, budget filtering, and total-cost calculation.

An autouse fixture in `conftest.py` resets shared model and catalog caches between tests, which is what keeps the suite deterministic when tests run in any order. Project documentation states a target of **~85% coverage**; that figure is a stated target rather than a coverage number reproduced independently for this report, and re-running `pytest --cov=src --cov=api --cov-report=term-missing` is the fastest way to confirm it before submission.

8. Deployment & Infrastructure

Four deployment-related components ship with the repository. Each is functional in isolation; none has yet been exercised as a live, connected system.

Component	Asset	Status	Notes
Docker	docker/Dockerfile	READY	Builds and runs the API image; bakes in whatever is present under artifacts/ at build time.
MLflow	docker-compose.mlflow.yml	PARTIAL	Tracking code is implemented and verified; compose starts a server on port 5001, but no model has been registered in a live registry yet.
Kubernetes	kubernetes/*.yaml	CONFIGURED	Namespace, ConfigMap, Deployment (2 replicas), and Service manifests are valid but have not been applied to a real cluster.
Airflow	airflow/dags/voyage_model_pipeline.py	TEMPLATE	Validate → register → validate-service → deployment-ready DAG exists but is not wired to a running scheduler.

READY

build/run verified

PARTIAL

works locally, not yet live

CONFIGURED / TEMPLATE

written but unexercised

The practical path from here to a live deployment is: start the MLflow server and run [scripts/track_models.py](#) to populate the registry, then `kubectl apply -f kubernetes/` against a real cluster, then connect the Airflow DAG to that same MLflow instance. None of these three steps depends on changing application code — Section 11 sequences them into a roadmap.

9. Repository & Code Metrics

The figures below are computed directly from the repository's own file listing, so they can be reproduced by anyone with the source tree.

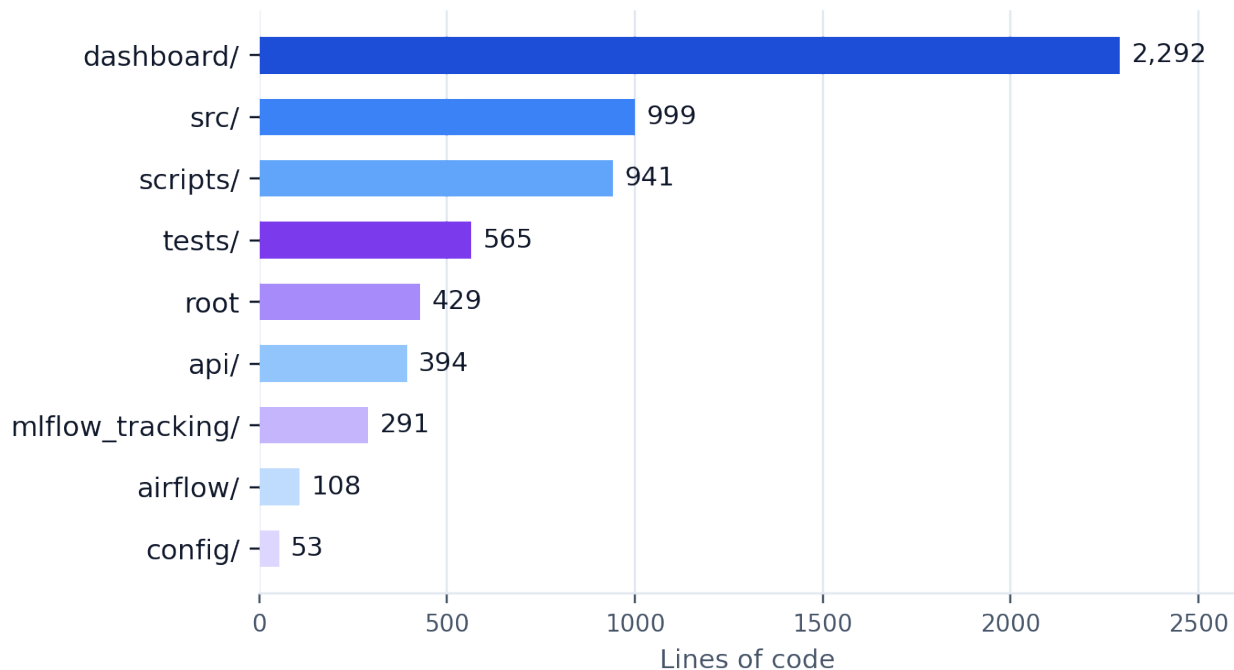


Figure 9.1 — Lines of Python code by top-level directory.

File composition

Type	Count
Python modules	42
Markdown documentation	18
YAML / YML manifests	6
CSV datasets	3
Other (config, notebook, assets)	15
Total	84

Repository footprint (~44.7 MB)

Contents	Approx. size
Source code + documentation	~227 KB (<1%)
Bundled CSV datasets	~27 MB
Model / catalog artifacts	~18 MB
Training notebook + logo asset	~2.1 MB

Documentation density is a notable feature of this repository: 18 Markdown files means nearly every top-level directory carries its own [README.md](#), which is why Sections 3 and 8 of this report could be written directly from the repository rather than from tribal knowledge. The flip side, visible above, is that over 99% of the repository's weight is data and model artifacts rather than code — the reason Update.md already flags external object storage as a future consideration (Risk #4).

10. Risk Register & Known Limitations

The nine items below are carried over from the project's own [Update.md](#), with an impact assessed for this report so they can be prioritized rather than read as a flat list.

#	Item	Impact	Recommended action
1	MLflow registry not live	MEDIUM	Run the bundled compose server and scripts/track_models.py to register the flight, gender, and recommendation models.
2	Model-flavour logging needs a healthy Python env	MEDIUM	Use the Linux CI runner or the Docker MLflow image rather than the affected Windows/Python 3.13 environment for flavour logging and registry serving.
3	Flight-price artifact is git-ignored	HIGH	Decide and document whether to commit the artifact for submission or inject it at build/deploy time; without it, /api/predict cannot serve real predictions on a fresh clone.
4	Raw datasets bundled (~27 MB)	LOW	Fine at current scale; plan a move to external/object storage if the repository grows further.
5	Gender model accuracy limited by label noise & fairness concerns	HIGH	Reframe to a 2-class task, source an external name-to-gender reference, or add features beyond name/age — and reconsider whether the feature should ship at all (Section 5.2).
6	Kubernetes not deployed to a live cluster	MEDIUM	Run <code>kubectl apply -f kubernetes/</code> against a real cluster; add an Ingress and an HPA policy.
7	Objectives 5 & 6 undefined	LOW	Confirm the missing objectives with the project stakeholder before final submission.
8	Airflow DAG is an unscheduled template	MEDIUM	Wire the DAG to real training steps and a running MLflow server if scheduled retraining is required.
9	Observability is minimal	HIGH	Add a /metrics endpoint plus per-model health/quality checks before any production exposure.

11. Roadmap

The nine risk items above resolve naturally into three phases, ordered so that each phase only depends on work already completed in the one before it.

Phase 1 — Immediate (before submission / next sprint)

- Standardize the supported runtime on Python 3.10 or 3.11 and run the full `pytest` suite in CI on that version.
- Decide how the flight-price artifact is injected into CI and deployment images (Risk #3).
- Start the MLflow server in the deployment environment and register all three model assets (Risk #1).

Phase 2 — Short-term

- Apply the Kubernetes manifests to a real cluster; add an Ingress and an HPA policy (Risk #6).
- Connect MLflow in-cluster so served predictions can be traced back to a registered model version.
- Confirm objectives 5 and 6 with the project stakeholder and close out the brief (Risk #7).

Phase 3 — Medium-term

- Define and implement monitoring signals for API health, model quality, and data drift, with alerting (Risk #9).
- Resolve the gender-classification fairness and accuracy concerns before any wider exposure (Risk #5).
- Connect the Airflow DAG to real training steps and the live MLflow server for scheduled retraining (Risk #8).

12. Conclusion

Voyage Analytics demonstrates a complete, working slice of an MLOps platform: three trained models, a validated API in front of them, an interactive dashboard, and the packaging and orchestration assets a production rollout would build on. The layering established in Section 3 — contracts separated from services, services separated from model loading — is what makes the system testable today and extensible tomorrow, and the 32-test suite is direct evidence that the separation holds up in practice.

What remains is operational rather than architectural: registering models in a live MLflow instance, applying the Kubernetes manifests to a real cluster, scheduling the Airflow retraining DAG, and adding the observability a production system needs. Section 11 sequences that work into three phases, and Section 10 gives each open item an assessed priority so it can be planned rather than discovered.

A. Environment & Configuration Reference

Runtime behavior is controlled entirely through environment variables, loaded once by `config/settings.py` into a single `settings` object.

Variable	Default	Role
<code>MODEL_PATH</code>	<code>artifacts/ flight_price_pipeline.joblib</code>	Must be present for real flight-price inference.
<code>GENDER_MODEL_PATH</code>	<code>artifacts/ gender_model.joblib</code>	Loaded by the gender service.
<code>HOTELS_CATALOG_PATH</code>	<code>artifacts/hotel_catalog.json</code>	Recommendation catalog consumed by the recommendation service.
<code>HOTELS_DATA_PATH</code>	<code>artifacts/data/hotels.csv</code>	Source data for catalog / training workflows.
<code>USERS_DATA_PATH</code>	<code>artifacts/data/users.csv</code>	Source data for catalog / training workflows.
<code>API_HOST</code>	<code>0.0.0.0</code>	Network bind address for the Flask service.
<code>API_PORT</code>	<code>5000</code>	Network bind port for the Flask service.
<code>MLFLOW_TRACKING_URI</code>	<code>http://localhost:5000 (settings) / :5001 (compose)</code>	Port alignment must be checked when the API and MLflow run together.

B. Pinned Dependencies

Core dependencies are version-pinned in `requirements.txt`, with MLflow and developer tooling separated into `requirements-dev.txt`. Python 3.10 or 3.11 is the recommended runtime — Python 3.13 has documented native-package issues on Windows affecting torch, pyarrow, and skops/XGBoost serialization.

Package	Version	Purpose
<code>flask</code>	3.0.3	Web framework.
<code>joblib</code>	1.4.2	Model artifact (de)serialization.
<code>xgboost</code>	2.0.3	Flight-price regression.
<code>scikit-learn</code>	1.7.2	Pinned to verify clean MLflow pickle serialization of the XGBoost pipeline and load the bundled artifact without issues.
<code>pydantic</code>	2.7.4	Request/response validation.
<code>pandas</code>	2.2.2	Data handling.
<code>requests</code>	2.32.3	Dashboard → API HTTP client.
<code>streamlit</code>	1.36.0	Dashboard framework.
<code>pytest / pytest-cov</code>	8.2.2 / 5.0.0	Test suite and coverage.
<code>mypy</code>	1.10.1	Static type checking.

C. Repository Structure

Top-level layout, condensed to two levels for reference. A per-file inventory with checksums is retained in the repository's own generated report and is omitted here.

```
./
├─ README.md, Update.md, generate_project_report.py, pyproject.toml
├─ requirements.txt, requirements-dev.txt
├─ .github/workflows/           → CI pipeline (test + Docker build)
├─ airflow/dags/                → retraining orchestration template
├─ api/                         → Flask app factory + blueprints
├─ artifacts/                   → model artifacts, catalog, data/
├─ assets/                      → static assets (logo)
├─ backend/                     → ML-team handoff pickle
├─ config/                      → environment-driven settings
├─ dashboard/                   → Streamlit application
├─ docker/                      → Dockerfile + MLflow compose
├─ kubernetes/                  → namespace, configmap, deployment, service
├─ mlflow_tracking/             → ModelTracker + tracking utilities
├─ notebook/                    → original Colab training notebook
├─ scripts/                     → train / build / track / validate CLIs
├─ src/
│   ├── features/                → shared feature engineering
│   ├── model/                   → cached artifact loaders
│   ├── schemas/                 → Pydantic contracts
│   └── services/                → prediction, gender, recommendation, search
└─ tests/                       → pytest suite + fixtures
```